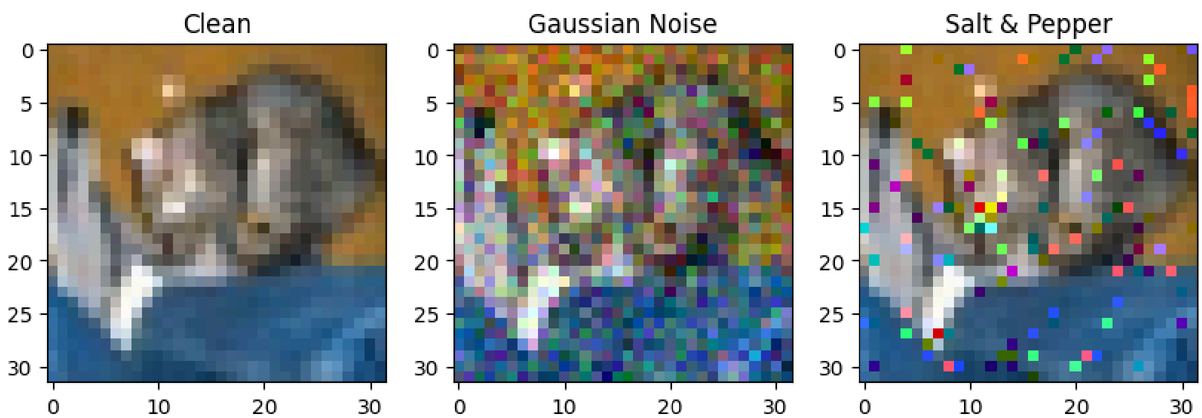
In [1]:

```python
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
import numpy as np
import matplotlib.pyplot as plt

# Load data
(x_train, _), (x_test, _) = cifar10.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

print("Data shape:", x_train.shape)

# Function to add noise
def add_gaussian_noise(images, sigma=0.1):
    noise = np.random.normal(0, sigma, images.shape)
    noisy = np.clip(images + noise, 0.0, 1.0)
    return noisy

def add_salt_pepper_noise(images, amount=0.05):
    noisy = images.copy()
    salt = np.random.random(images.shape) < amount/2
    pepper = np.random.random(images.shape) < amount/2
    noisy[salt] = 1.0
    noisy[pepper] = 0.0
    return noisy

# Create noisy versions
x_train_gauss = add_gaussian_noise(x_train, sigma=0.1)
x_test_gauss = add_gaussian_noise(x_test, sigma=0.1)

x_train_sp = add_salt_pepper_noise(x_train, amount=0.05)
x_test_sp = add_salt_pepper_noise(x_test, amount=0.05)

# Visualization example
plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.imshow(x_test[0]); plt.title('Clean')
plt.subplot(1,3,2); plt.imshow(x_test_gauss[0]); plt.title('Gaussian Noise')
plt.subplot(1,3,3); plt.imshow(x_test_sp[0]); plt.title('Salt & Pepper')
plt.show()
```

```
2026-05-03 13:19:44.336713: I external/local_tsl/tsl/cuda/cudart_stub.cc:32]
Could not find cuda drivers on your machine, GPU will not be used.
2026-05-03 13:19:44.409938: I external/local_tsl/tsl/cuda/cudart_stub.cc:32]
Could not find cuda drivers on your machine, GPU will not be used.
2026-05-03 13:19:44.565403: E external/local_xla/xla/stream_executor/cuda/cu
da_fft.cc:479] Unable to register cuFFT factory: Attempting to register fact
ory for plugin cuFFT when one has already been registered
2026-05-03 13:19:44.762315: E external/local_xla/xla/stream_executor/cuda/cu
da_dnn.cc:10575] Unable to register cuDNN factory: Attempting to register fa
ctory for plugin cuDNN when one has already been registered
2026-05-03 13:19:44.762875: E external/local_xla/xla/stream_executor/cuda/cu
da_blas.cc:1442] Unable to register cuBLAS factory: Attempting to register f
actory for plugin cuBLAS when one has already been registered
2026-05-03 13:19:44.986281: I tensorflow/core/platform/cpu_feature_guard.cc:
210] This TensorFlow binary is optimized to use available CPU instructions i
n performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
2026-05-03 13:19:46.923121: W tensorflow/compiler/tf2tensorrt/utils/py_util
s.cc:38] TF-TRT Warning: Could not find TensorRT
```

```
Data shape: (50000, 32, 32, 3)
```



In [2]:

```python
def build_dae():
    input_img = tf.keras.Input(shape=(32, 32, 3))

    # Encoder
    x = tf.keras.layers.Conv2D(64, (3,3), activation='relu', padding='same')
    x = tf.keras.layers.MaxPooling2D((2,2), padding='same')(x)
    x = tf.keras.layers.Conv2D(128, (3,3), activation='relu', padding='same'
    encoded = tf.keras.layers.MaxPooling2D((2,2), padding='same')(x)

    # Decoder (using Transposed Convolution)
    x = tf.keras.layers.Conv2DTranspose(128, (3,3), activation='relu', paddi
    x = tf.keras.layers.UpSampling2D((2,2))(x)
    x = tf.keras.layers.Conv2DTranspose(64, (3,3), activation='relu', paddir
    x = tf.keras.layers.UpSampling2D((2,2))(x)
    decoded = tf.keras.layers.Conv2D(3, (3,3), activation='sigmoid', padding

    autoencoder = tf.keras.Model(input_img, decoded)
    autoencoder.compile(optimizer='adam', loss='mse')
    return autoencoder
```

```python
dae = build_dae()
dae.summary()
```

Model: "functional"

Layer (type)	Output Shape	Par
input_layer (InputLayer)	(None, 32, 32, 3)	
conv2d (Conv2D)	(None, 32, 32, 64)	1
max_pooling2d (MaxPooling2D)	(None, 16, 16, 64)	
conv2d_1 (Conv2D)	(None, 16, 16, 128)	73
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 128)	
conv2d_transpose (Conv2DTranspose)	(None, 8, 8, 128)	147
up_sampling2d (UpSampling2D)	(None, 16, 16, 128)	
conv2d_transpose_1 (Conv2DTranspose)	(None, 16, 16, 64)	73
up_sampling2d_1 (UpSampling2D)	(None, 32, 32, 64)	
conv2d_2 (Conv2D)	(None, 32, 32, 3)	1

Total params: 298,755 (1.14 MB)

Trainable params: 298,755 (1.14 MB)

Non-trainable params: 0 (0.00 B)

In [4]:
```python
# Train on Gaussian noise
print("Training DAE on Gaussian Noise...")
dae_gauss = build_dae()
dae_gauss.fit(x_train_gauss, x_train,
              epochs=5,
              batch_size=64,
              validation_data=(x_test_gauss, x_test),
              verbose=1)

# Train on Salt & Pepper noise
print("Training DAE on Salt & Pepper Noise...")
dae_sp = build_dae()
dae_sp.fit(x_train_sp, x_train,
           epochs=5,
           batch_size=64,
           validation_data=(x_test_sp, x_test),
           verbose=1)
```

```
Training DAE on Gaussian Noise...
2026-05-03 13:31:25.536654: W external/local_tsl/tsl/framework/cpu_allocator
_impl.cc:83] Allocation of 614400000 exceeds 10% of free system memory.
2026-05-03 13:31:28.165922: W external/local_tsl/tsl/framework/cpu_allocator
_impl.cc:83] Allocation of 614400000 exceeds 10% of free system memory.
```

```
Epoch 1/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 235s 297ms/step - loss: 0.0074 - val_loss: 0.00
45
Epoch 2/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 232s 297ms/step - loss: 0.0041 - val_loss: 0.00
38
Epoch 3/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 219s 280ms/step - loss: 0.0034 - val_loss: 0.00
31
Epoch 4/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 221s 283ms/step - loss: 0.0030 - val_loss: 0.00
28
Epoch 5/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 219s 280ms/step - loss: 0.0028 - val_loss: 0.00
28
Training DAE on Salt & Pepper Noise...
2026-05-03 13:50:15.874681: W external/local_tsl/tsl/framework/cpu_allocator
_impl.cc:83] Allocation of 614400000 exceeds 10% of free system memory.
Epoch 1/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 223s 282ms/step - loss: 0.0077 - val_loss: 0.00
47
Epoch 2/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 223s 285ms/step - loss: 0.0042 - val_loss: 0.00
37
Epoch 3/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 229s 293ms/step - loss: 0.0034 - val_loss: 0.00
31
Epoch 4/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 233s 298ms/step - loss: 0.0030 - val_loss: 0.00
27
Epoch 5/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 235s 300ms/step - loss: 0.0026 - val_loss: 0.00
25
```

Out[4]:
```
<keras.src.callbacks.history.History at 0x71fcec10a750>
```

In [6]:
```python
def plot_denoising_results(model, noisy_images, clean_images, n=5):
    plt.figure(figsize=(15, 6))
    for i in range(n):
        # Noisy
        plt.subplot(3, n, i+1)
        plt.imshow(noisy_images[i])
        plt.title('Noisy')
        plt.axis('off')

        # Reconstructed
        pred = model.predict(np.expand_dims(noisy_images[i], 0), verbose=0)[
        plt.subplot(3, n, i+1+n)
        plt.imshow(pred)
        plt.title('Denoised')
        plt.axis('off')

        # Original
        plt.subplot(3, n, i+1+2*n)
        plt.imshow(clean_images[i])
        plt.title('Original')
```

```
        plt.axis('off')
    plt.tight_layout()
    plt.show()

# Example for Gaussian
plot_denoising_results(dae_gauss, x_test_gauss[:5], x_test[:5])
# Example for Salt and Pepper
plot_denoising_results(dae_sp, x_test_sp[:5], x_test[:5])
```